

Guion de Clase: Dominando el Patrón Composite

Objetivo: Devolución del Examen Parcial, análisis de errores conceptuales y preparación para el recuperatorio.

Tono: Firme, pedagógico, directo y motivador.

Filmina 1: Portada

"Buenas noches a todos. Hoy vamos a hacer una pausa necesaria sobre el parcial de la semana pasada. Analizando las entregas, identifiqué tres trabas puntuales que se repitieron en la mayoría de los trabajos. El objetivo de hoy no es reprochar errores, sino desarmar exactamente qué pasó, entender por qué el patrón Composite era la única respuesta natural al problema y dejar las bases 100% claras para el recuperatorio."

Filmina 2: Diagnóstico del Examen

"Fueron tres los tropiezos principales:

Primero: la elección del patrón. Teniendo solo cuatro opciones en el programa, varios dudaron o eligieron patrones creacionales.

Segundo: se marearon entre lo simple y lo compuesto, rompiendo el polimorfismo al intentar modelar las colecciones.

Y tercero: cuando quisieron calcular los costos o interactuar desde la consola, chocaron contra la recursividad o cayeron en referencias circulares que tildaron el programa. Vamos a atacar estos tres puntos uno por uno."

Filmina 3: Criterio de Decisión entre Familias GoF

"Muchos leyeron 'presupuesto' o 'planificación' y pensaron automáticamente en 'crear objetos', yéndose a Factory Method o Abstract Factory. Otros vieron que al agregar tareas había que recalcular y pensaron en notificaciones con Observer.

Recuerden la taxonomía básica: Factory resuelve el CÓMO se instancian los objetos; Observer resuelve CÓMO se comunican cuando cambia el estado. Pero el parcial no les pedía familias de familias ni eventos: les pedía representar una obra que contiene etapas y tareas. Es un problema PURAMENTE ESTRUCTURAL. Y de los cuatro patrones vistos, el único estructural es Composite."

Filmina 4: El Principio de Uniformidad

"La definición del Gang of Four se resume en una sola idea: tratar a los objetos individuales y a

sus composiciones de manera idéntica.

¿Qué significa esto en la práctica? Que al cliente no le tiene que importar si lo que tiene entre manos es pintar una pared o la construcción entera del edificio. En ambos casos, le pide `CalcularCosto()` y el objeto sabe responder. Si tienen que preguntar con un `if` qué tipo de objeto es antes de operarlo, rompieron la uniformidad."

Filmina 5: Roles y Relaciones del Patrón (Estructura Canónica)

*"Miren el diagrama canónico en pantalla, tiene cuatro piezas esenciales:

1. El Componente: la interfaz o clase abstracta que define las operaciones comunes.
2. La Hoja (Leaf): el elemento simple que no tiene hijos y hace el trabajo real.
3. El Compuesto (Composite): el contenedor que guarda una lista de componentes y DELEGA el trabajo a sus hijos. Fíjense en la flecha de agregación: no guarda Hojas, guarda Componentes.
- Y 4. El Cliente: que interactúa exclusivamente a través de la abstracción, sin atarse a las clases concretas."

Filmina 6: Error Típico N° 1 - El Anti-Patrón de las Dos Listas

"Este fue el error conceptual más grave del examen. Muchos hicieron una clase `Etapa` con dos listas: `List<Tarea>` y `List<Etapa>`.

¿Por qué está mal? Porque destruye el polimorfismo. Al hacer eso, se ven obligados a escribir dos bucles separados para sumar costos y pierden la recursividad natural. En `Composite` hay UNA SOLA lista: `List<ComponenteProyecto>`. Al compuesto no le importa si lo que tiene adentro es una tarea simple u otra etapa; para él, todos son componentes."

Filmina 7: Mapeo de Dominio - Tarea vs. Etapa

"Bajemos esto al examen:

La Hoja es la Tarea: tiene costo de materiales, costo de mano de obra y calcula su total sumando ambos. Punto. No delega nada porque no tiene hijos.

El Compuesto es la Etapa: no tiene materiales ni mano de obra propios. Su costo es la suma de los costos de sus elementos hijos.

Ambos heredan de `ComponenteProyecto`. Es así de simple y directo."

Filmina 8: Transparencia vs. Seguridad

"Acá surgió la duda: ¿dónde ponemos los métodos `Agregar()` y `Quitar()`?

Si los ponemos en la clase base, ganamos transparencia: el cliente los ve en todos lados, pero en `Tarea` tendríamos que lanzar una excepción porque una tarea no puede tener hijos.

Por eso, para este examen el enfoque recomendado era el de SEGURIDAD: declarar Agregar() y Quitar() únicamente en Etapa. Las hojas solo hacen su trabajo y evitamos excepciones en tiempo de ejecución."

Filmina 9: La Elegancia de la Delegación (C#)

"Miren lo que queda en el código de la clase Etapa:

```
return _elementos.Sum(e => e.CalcularCosto());
```

Una sola línea de LINQ. Cero if, cero operadores is, cero switch.

Si e es una Tarea, sumará sus materiales y mano de obra. Si e es otra Etapa, llamará internamente a su propio Sum(). El compilador y el despacho dinámico resuelven la recursividad por ustedes."

Filmina 10: La Trampa de los Grafos Cíclicos

"Ahora vamos al tercer problema: los cuelgues del sistema y los StackOverflowException.

Un Composite modela un ÁRBOL, y un árbol por definición matemática es ACÍCLICO.

Si ustedes le permiten al usuario en la consola tomar una etapa y agregarla adentro de sí misma, o adentro de una etapa que es su hija, transformaron el árbol en un grafo con ciclos. Al llamar a CalcularCosto(), la recursividad entra en un bucle infinito."

Filmina 11: Blindaje Defensivo contra Ciclos

"¿Cómo se previene esto en código? Con programación defensiva en el método Agregar().

Como mínimo, validamos dos cosas: primero, que el componente a agregar no sea this. Y segundo, si manejan referencia al padre, validar que no estemos intentando agregar un nodo ancestro. Si la condición no se cumple, lanzamos una excepción clara antes de corromper la estructura."

Filmina 12: Construcción Segura desde Consola

*"La consigna pedía interacción con el usuario, y muchos se perdieron armando menús complicadísimos.

La interacción no requería un laberinto de pantallas. Alcanzaba con tres pasos ordenados:

1. Seleccionar la etapa contenedora de destino.
2. Pedir los datos por teclado e instanciar la nueva Tarea o Etapa.
3. Agregarla a la colección mediante el método seguro.

Mantengan la UI simple para no desenfocarse del diseño de clases."*

Filmina 13: Diagrama de Clases del Examen

"Este es el diagrama UML que debían entregar:

Arriba, la clase abstracta `ComponenteProyecto` con su método abstracto `CalcularCosto()`.

Abajo, `Tarea` implementando el cálculo con sus atributos directos.

Al lado, `Etapa`, con una relación de agregación recursiva hacia `ComponenteProyecto`, con multiplicidad `*`.

Comparen esto con lo que entregaron la semana pasada y vean qué flechas o clases les sobraban o les faltaban."

Filmina 14: Checklist para el Recuperatorio

*"Para el recuperatorio, antes de entregar, verifiquen estos tres puntos clave:

1. Polimorfismo real: ¿tienen una sola lista de tipo base en el compuesto? Si tienen listas de tipos concretos, corrijan.
2. Delegación limpia: ¿su método de cálculo delega directamente en los hijos sin casteos ni `if`?
3. Integridad del árbol: ¿están controlando que no se agreguen elementos sobre sí mismos? Si estos tres ítems están en verde, el ejercicio está aprobado."*

Filmina 15: Cierre

"Los patrones de diseño no son recetas para memorizar; son vocabulario para estructurar soluciones limpias y desacopladas. `Composite` convierte estructuras jerárquicas complejas en llamadas elegantes y mantenibles.

Repasen este material, revisen su examen con este criterio y abrimos el espacio para dudas puntuales sobre el código de cada uno."